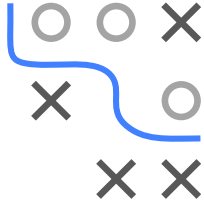
Optimization in Machine Learning

Advanced first order methods
In practice

Learning goals

- Optimizers are pre-implemented in major frameworks
- Minimal training loop in PyTorch
- Hyperparameters and sensible defaults

OPTIMIZERS ARE ALREADY IMPLEMENTED

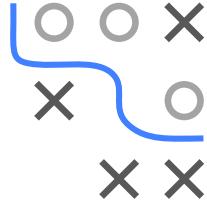


- All methods we saw (GD, SGD, . . . , Adam) are pre-implemented in major ML frameworks
- You rarely implement them yourself – just configure and call
- This reflects how generic and effective they are

Python PyTorch (`torch.optim`), JAX (`optax`), TensorFlow / Keras (`keras.optimizers`)

R available via `{torch}` and `{keras}`; base `optim()` instead provides quasi-Newton methods (see Chapter 5)

MINIMAL EXAMPLE: ADAM IN PYTORCH



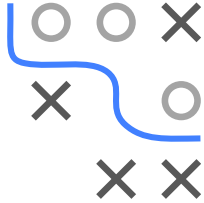
```
# model: a torch.nn.Module; loss_fn: e.g. MSELoss

opt = torch.optim.Adam(model.parameters(), lr=1e-3)

for x, y in dataloader:           # mini-batches
    opt.zero_grad()               # reset gradients
    loss = loss_fn(model(x), y)   # forward pass
    loss.backward()               # backprop -> .grad
    opt.step()                    # Adam update
```

- The same four calls drive every optimizer: swap `Adam` for `SGD`, `AdaGrad`, ...

HYPERPARAMETERS: DEFAULTS USUALLY WORK



- These methods introduce hyperparameters: step size α, stability constant β, decay rates $\rho_1, \rho_2, \ldots$
- **But:** Defaults are usually strong – e.g. Adam with $\alpha = 0.001$, $\rho_1 = 0.9$, $\rho_2 = 0.999$, $\beta = 10^{-8}$ works across many tasks
- $\Rightarrow$ Can be applied effectively without extensive tuning (although tuning can pay off)

- Main knob worth tuning: the **step size** (and its schedule)